

Proyecto 2

Análisis Semántico de Compiscript

Descripción

Esta actividad corresponde a la fase de análisis semántico en la construcción de un compilador para **Compiscript**, un lenguaje basado en un subconjunto de TypeScript. Se debe implementar un analizador sintáctico a partir de la gramática oficial en ANTLR y extenderlo con **acciones semánticas**, construyendo un árbol sintáctico con representación visual y recorriéndolo mediante *Listeners* o *Visitors* de ANTLR para validar las reglas semánticas del lenguaje.

- **Analizador Sintáctico:** Basado en la gramática de Compiscript (ANTLR), reutilizando o extendiendo el trabajo de la fase anterior.
- **Análisis Semántico:** Verificación de tipos, ámbitos, funciones, control de flujo, clases y estructuras de datos sobre el árbol sintáctico generado.
- **Tabla de Símbolos:** Estructura que acompaña todas las fases del compilador, manejando entornos anidados (global, función, clase, bloque).

Modalidad: Grupos de 3 integrantes.

Objetivos

Generales:

- Implementar un analizador semántico funcional para el lenguaje Compiscript.
- Aplicar los conceptos de sistemas de tipos, ámbitos y entornos de ejecución sobre un árbol sintáctico real.
- Diseñar una tabla de símbolos capaz de sostener las fases posteriores del compilador.

Específicos:

- Implementar el analizador sintáctico de Compiscript utilizando ANTLR (u otra herramienta similar).
- Construir el árbol sintáctico de Compiscript con representación visual.
- Recorrer el árbol mediante Listeners o Visitors de ANTLR para evaluar las reglas semánticas.

- Implementar una batería de pruebas que valide casos exitosos y fallidos para cada regla semántica.
- Desarrollar un IDE que permita escribir y compilar código Compiscript.

Especificaciones

El analizador semántico debe validar, como mínimo, las siguientes reglas:

▪ Sistema de Tipos:

- Verificación de tipos en operaciones aritméticas (+, -, *, /) — operandos de tipo `integer` o `float`.
- Verificación de tipos en operaciones lógicas (&&, ||, !) — operandos de tipo `boolean`.
- Compatibilidad de tipos en comparaciones (==, !=, <, <=, >, >=).
- Verificación de tipos en asignaciones, coincidiendo con el tipo declarado de la variable.
- Inicialización obligatoria de constantes (`const`) en su declaración.
- Verificación de tipos en listas y estructuras.

▪ Manejo de Ámbito:

- Resolución de nombres según ámbito local o global.
- Error por uso de variables no declaradas.
- Prohibición de redeclaración de identificadores en el mismo ámbito.
- Control de acceso a variables en bloques anidados.
- Creación de un nuevo entorno de símbolos por cada función, clase y bloque.

▪ Funciones y Procedimientos:

- Validación de número y tipo de argumentos en llamadas a funciones (coincidencia posicional).
- Validación del tipo de retorno respecto al tipo declarado.
- Soporte para funciones recursivas.
- Soporte para funciones anidadas y closures, capturando variables del entorno de definición.
- Detección de redeclaración de funciones con el mismo nombre.

▪ Control de Flujo:

- Las condiciones de `if`, `while`, `do-while`, `for` y `switch` deben ser de tipo `boolean`.
- `break` y `continue` solo permitidos dentro de bucles.
- `return` solo permitido dentro del cuerpo de una función.

▪ Clases y Objetos:

- Validación de existencia de atributos y métodos accedidos mediante `.`
 - Verificación de la correcta invocación del constructor.
 - Manejo correcto de `this` dentro del ámbito de la clase.
- **Listas y Estructuras de Datos:**
 - Verificación del tipo de los elementos de una lista.
 - Validación de índices en el acceso a listas.
 - **Generales:**
 - Detección de código muerto (instrucciones después de `return`, `break`, etc.).
 - Verificación de sentido semántico en expresiones (por ejemplo, no multiplicar funciones).
 - Validación de declaraciones duplicadas (variables, parámetros).

Entrada:

- Archivo fuente de Compiscript con extensión `.cps`.

Salida:

- Representación visual del árbol sintáctico generado.
- Reporte de errores semánticos encontrados (tipo, ámbito, funciones, control de flujo, clases, listas).
- Estado de la tabla de símbolos por cada entorno (global, función, clase, bloque).

Funcionamiento del Programa

1. El usuario carga o escribe un archivo `.cps` a través del IDE.
2. El analizador léxico y sintáctico (ANTLR) construye el árbol sintáctico, mostrando su representación visual.
3. Un Listener o Visitor recorre el árbol aplicando las reglas semánticas descritas, construyendo y consultando la tabla de símbolos en cada entorno.
4. El programa reporta los errores semánticos encontrados, con su tipo y ubicación.
5. El usuario puede ejecutar la batería de pruebas para validar casos exitosos y fallidos de cada regla semántica.

Entregables

- Un repositorio de GitHub, con commits individuales que evidencien claramente la contribución de cada integrante (no se permite compartir commits en conjunto).
- Batería de pruebas para cada regla semántica (casos exitosos y fallidos), presente y funcional al momento de la evaluación.
- Documentación de la arquitectura de la implementación y documentación de cómo ejecutar el compilador.
- IDE funcional que permita escribir y compilar código Compiscript.

Evaluación

Requisitos para Calificación

Para poder optar a calificación, el programa debe funcionar correctamente el día de la presentación. El proyecto debe estar correctamente documentado y organizado.

Componente	Descripción	Puntos
IDE	Entorno funcional que permite escribir, cargar y compilar código Compiscript	15 pts
Analizador Sintáctico y Semántico	Validación de reglas semánticas y sistema de tipos, con árbol sintáctico visual y batería de pruebas	60 pts
Tabla de Símbolos	Manejo correcto de entornos anidados (global, función, clase, bloque) a lo largo de las fases del compilador	25 pts
Total		100 pts